

claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-a6712e11fbb5c8355.jsonl`  
- SHA-256: `2a50f7059237e1d4dcdfce2cf72b5f8edbbdef3c0a19fb75ba66087d40b8e64673`  
- Source modified: `2026-07-07T04:17:12+00:00`  
- Imported at: `2026-07-08T16:00:26+00:00`  
- project: `wf_ddb0b3fd-b40`  
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`
```

Transcript

1. user (2026-07-07T04:14:53.571Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:

- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:

- file: tests/test_wasb_nvdec_resident.py
- line: 293
- summary: The hardware parity gate cannot run under its own documented invocation: the in-test import `from backend.pipeline.detectors import native\_wasb\_runner as nwr` pulls backend.config.models -> pydantic, which is not installed in the wasb conda env built by deploy/cloudrun/Dockerfile or deploy/digitalocean/gpu_setup.sh (dep list: torch/torchvision/PyNvVideoCodec/hydra-core/omegaconf/pandas/opencv/pillow/scipy/tqdm/pyyaml/matplotlib/numpy ? none depend on pydantic). The import exists only to read nwr._INFER_PY, a path derivable from the already-imported stdlib-only wi module (osp.join(osp.dirname(wi.__file__), 'wasb_infer.py')). Confirmed by simulation: blocking pydantic makes native_wasb_runner import raise ModuleNotFoundError while wasb_infer imports fine.
- failure scenario: On the GPU box, following the test's own docstring (conda activate wasb; pip install pytest; WASB_REPO_DIR=... WASB_WEIGHTS=... WASB_PARITY_VIDEO=... python -m pytest tests/test_wasb_nvdec_resident.py -k parity): all skip guards pass (torch, CUDA, PyNvVideoCodec present), then the test ERRORS with ModuleNotFoundError: No module named 'pydantic' ? the #506

flip-gate parity check never actually runs as documented.

- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario actually happens in the code as written. Default to isReal=false if the scenario cannot occur, is already guarded, or rests on a misreading.

2. user (2026-07-07T04:15:02.629Z)

```
1      """GPU-resident NVDEC decode backend (#506) ? the parts testable off-GPU, plus the
2      on-box parity gate.
3
4      THE lesson this file locks (issue #506): the GPU resize MUST replicate WASB's
5      ``cv2.warpAffine(get_affine_transform(...), INTER_LINEAR)`` exactly. A naive
6      ``F.interpolate`` resize samples at half-pixel-convention positions ? a constant
7      sub-pixel shift versus cv2's integer-pixel-center affine ? which tips the sigmoid
8      past ``score_threshold`` on marginal frames and silently over-detects (rally F1
9      0.58 ? 0.07 on the hard golden clip GX010128, while the easy clip masked it at 98%
10     trajectory agreement). The grid-parity tests below fail if anyone "simplifies" the
11     affine ``grid_sample`` back to a plain resize.
12
13     The trajectory-parity contract itself (``nvdec_resident`` ? ``cpu`` on a real clip)
14     is a HARDWARE test ? it needs a CUDA GPU, PyNvVideoCodec, the WASB repo, weights and
15     a fixture clip ? so it skips everywhere else (CI is CPU-only). See
16     ``test_nvdec_resident_trajectory_matches_cpu_on_fixture`` for how to run it on the box.
17     """
18
19     import csv
20     import importlib.util
21     import os
22     import os.path as osp
23     import sys
24
25     import numpy as np
26     import pytest
27
28     from backend.pipeline.detectors import wasb_infer as wi
29
30
31     # ----- #
32     # WASB's exact affine geometry, replicated for the tests (utils/image.py
33     # ``get_affine_transform`` with rot=0: center = frame middle, scale = max(h, w),
34     # 3-point correspondence through cv2.getAffineTransform). The runtime path calls
35     # WASB's own function; the tests only need matrices of the same shape.
36     # ----- #
37     def _get_3rd_point(a, b):
38         direct = a - b
39         return b + np.array([-direct[1], direct[0]], dtype=np.float32)
40
41
42     def _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h):
43         """(forward, inverse) 2x3 matrices exactly as WASB's get_transform builds them."""
44         import cv2
45
46         c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
47         s = max(orig_h, orig_w) * 1.0
48         src = np.zeros((3, 2), dtype=np.float32)
49         dst = np.zeros((3, 2), dtype=np.float32)
50         src[0, :] = c
51         src[1, :] = c + np.array([0, -s * 0.5], dtype=np.float32)
52         dst[0, :] = [dst_w * 0.5, dst_h * 0.5]
53         dst[1, :] = dst[0, :] + np.array([0, -dst_w * 0.5], dtype=np.float32)
54         src[2, :] = _get_3rd_point(src[0, :], src[1, :])
55         dst[2, :] = _get_3rd_point(dst[0, :], dst[1, :])
```

```

56     fwd = cv2.getAffineTransform(np.float32(src), np.float32(dst))
57     inv = cv2.getAffineTransform(np.float32(dst), np.float32(src))
58     return fwd, inv
59
60
61 def _smooth_test_image(orig_w, orig_h, seed=506):
62     """Deterministic smooth float32 RGB image: smooth enough that both bilinear
63     implementations agree on identical sample positions, textured enough that a
64     sub-pixel sampling shift shows up loudly."""
65     import cv2
66
67     rng = np.random.default_rng(seed)
68     img = rng.random((orig_h, orig_w, 3)).astype(np.float32)
69     return cv2.GaussianBlur(img, (0, 0), sigmaX=2.0)
70
71
72 # ----- #
73 # The resize-fidelity lock (CPU-runnable: torch + cv2, no GPU / no WASB repo)
74 # ----- #
75 def test_affine_grid_matches_cv2_warppaffine():
76     """grid_sample over build_affine_grid == cv2.warpAffine on WASB's production
77     geometry (16:9 frame, the badminton 512x288-shaped target)."""
78     torch = pytest.importorskip("torch")
79     cv2 = pytest.importorskip("cv2")
80     import torch.nn.functional as F
81
82     orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36 # 16:9 ? 16:9, scale 3.0
83     img = _smooth_test_image(orig_w, orig_h)
84     fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
85
86     ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
87
88     grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
89     assert tuple(grid.shape) == (1, dst_h, dst_w, 2)
90     chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)
91     got = (
92         F.grid_sample(
93             chw, grid, mode="bilinear", align_corners=False, padding_mode="border"
94         )
95         .squeeze(0)
96         .permute(1, 2, 0)
97         .numpy()
98     )
99
100     diff = np.abs(got - ref)
101     # cv2 quantizes bilinear coefficients (INTER_BITS); anything beyond that noise
102     # floor means the geometry diverged. The naive-resize failure below is ~100x this.
103     assert diff.mean() < 1e-3, f"mean |?| {diff.mean():.2e} vs cv2.warpAffine"
104     assert diff.max() < 0.02, f"max |?| {diff.max():.2e} vs cv2.warpAffine"
105
106
107 def test_naive_interpolate_does_not_match_cv2_warppaffine():
108     """The #506 regression signature: F.interpolate is NOT the affine warp. If this
109     test ever fails (interpolate suddenly 'matches'), the reference setup is broken ?
110     it exists so the grid can never be quietly replaced by a plain resize."""
111     torch = pytest.importorskip("torch")
112     cv2 = pytest.importorskip("cv2")
113     import torch.nn.functional as F
114
115     orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36
116     img = _smooth_test_image(orig_w, orig_h)
117     fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
118
119     ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
120     chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)

```

```

121
122     grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
123     affine = (
124         F.grid_sample(
125             chw, grid, mode="bilinear", align_corners=False, padding_mode="border"
126         )
127         .squeeze(0)
128         .permute(1, 2, 0)
129         .numpy()
130     )
131     naive = (
132         F.interpolate(chw, size=(dst_h, dst_w), mode="bilinear", align_corners=False)
133         .squeeze(0)
134         .permute(1, 2, 0)
135         .numpy()
136     )
137
138     affine_err = np.abs(affine - ref).mean()
139     naive_err = np.abs(naive - ref).mean()
140     # The half-pixel-convention shift is a real, systematic error ? far above the
141     # affine grid's coefficient-quantization noise floor.
142     assert naive_err > 1e-3, f"naive resize unexpectedly matches cv2 ({naive_err:.2e})"
143     assert naive_err > 5 * affine_err, (
144         f"affine grid ({affine_err:.2e}) must beat naive resize ({naive_err:.2e}) "
145         f"decisively ? otherwise the #506 lesson is unlearned"
146     )
147
148
149     # ----- #
150     # Cache keying: nvdec detections are trajectory-level (not byte-level) equivalents
151     # of cpu ones, so the resumable cache must never be shared across backends.
152     # ----- #
153     def _manifest(**over):
154         m = {
155             "version": wi.CACHE_VERSION,
156             "frames_dir": "video:/x.mp4",
157             "weights": "/w.pth.tar",
158             "sport": "badminton",
159             "frames_in": 3,
160             "frames_total": 100,
161             "device": "cuda",
162         }
163         m.update(over)
164         return m
165
166
167     def test_cache_never_shared_across_decode_backends():
168         kw = dict(
169             frames_dir="video:/x.mp4",
170             weights="/w.pth.tar",
171             sport="badminton",
172             frames_in=3,
173             frames_total=100,
174             device="cuda",
175         )
176         # Pre-#506 manifests carry no decode_backend field == cpu-decoded: still valid
177         # for a cpu run (no invalidation on upgrade), NEVER for an nvdec run.
178         legacy = _manifest()
179         assert wi.manifest_compatible(legacy, **kw)
180         assert not wi.manifest_compatible(legacy, **kw, decode_backend="nvdec_resident")
181         # And an nvdec cache must not be replayed by a cpu run.
182         nv = _manifest(decode_backend="nvdec_resident")
183         assert wi.manifest_compatible(nv, **kw, decode_backend="nvdec_resident")
184         assert not wi.manifest_compatible(nv, **kw)
185

```

```

186
187 # ----- #
188 # CLI guards: the nvdec branch must fail loud BEFORE any torch/GPU work.
189 # ----- #
190 def test_cli_nvdec_resident_requires_video(monkeypatch, tmp_path):
191     monkeypatch.setattr(
192         sys,
193         "argv",
194         [
195             "wasb_infer.py",
196             "--decode-backend",
197             "nvdec_resident",
198             "--frames_dir",
199             str(tmp_path),
200             "--weights",
201             "w.pth.tar",
202             "--out",
203             str(tmp_path / "o.csv"),
204         ],
205     )
206     with pytest.raises(SystemExit, match="requires --video"):
207         wi.main()
208
209
210 def test_cli_nvdec_resident_requires_cuda_device(monkeypatch, tmp_path):
211     monkeypatch.setattr(
212         sys,
213         "argv",
214         [
215             "wasb_infer.py",
216             "--decode-backend",
217             "nvdec_resident",
218             "--video",
219             "v.mp4",
220             "--device",
221             "cpu",
222             "--weights",
223             "w.pth.tar",
224             "--out",
225             str(tmp_path / "o.csv"),
226         ],
227     )
228     with pytest.raises(SystemExit, match="cuda"):
229         wi.main()
230
231
232 def test_cli_rejects_unknown_decode_backend(monkeypatch, tmp_path, capsys):
233     monkeypatch.setattr(
234         sys,
235         "argv",
236         [
237             "wasb_infer.py",
238             "--decode-backend",
239             "gpu", # not a backend; argparse choices reject it
240             "--video",
241             "v.mp4",
242             "--weights",
243             "w.pth.tar",
244             "--out",
245             str(tmp_path / "o.csv"),
246         ],
247     )
248     with pytest.raises(SystemExit):
249         wi.main()
250     assert "invalid choice" in capsys.readouterr().err

```

```

251
252
253 # ----- #
254 # The #506 unit gate: nvdec_resident trajectory ? cpu trajectory on a real clip.
255 # HARDWARE test ? skips unless the box provides everything. Run on the GPU box
256 # (inside the torch-2.x wasb env, which has torch+cuda, PyNvVideoCodec, cv2):
257 #
258 # WASB_REPO_DIR=~/.models/WASB-SBDT \
259 # WASB_WEIGHTS=~/.models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
260 # WASB_PARITY_VIDEO=/path/to/short_fixture_clip.mp4 \
261 # python -m pytest tests/test_wasb_nvdec_resident.py -k parity -v
262 # ----- #
263 def _load_traj(path):
264     out = {}
265     with open(path, newline="") as f:
266         for row in csv.DictReader(f):
267             out[int(float(row["Frame"]))] = (
268                 int(float(row["Visibility"])),
269                 float(row["X"]),
270                 float(row["Y"]),
271             )
272     return out
273
274
275 def test_nvdec_resident_trajectory_matches_cpu_on_fixture(tmp_path):
276     repo = osp.expanduser(os.environ.get("WASB_REPO_DIR", ""))
277     weights = osp.expanduser(os.environ.get("WASB_WEIGHTS", ""))
278     video = osp.expanduser(os.environ.get("WASB_PARITY_VIDEO", ""))
279     if not (repo and weights and video):
280         pytest.skip("set WASB_REPO_DIR + WASB_WEIGHTS + WASB_PARITY_VIDEO to run")
281     for p in (osp.join(repo, "src"), weights, video):
282         if not osp.exists(p):
283             pytest.skip(f"missing {p}")
284     torch = pytest.importorskip("torch")
285     if not torch.cuda.is_available():
286         pytest.skip("needs a CUDA GPU")
287     pytest.importorskip("PyNvVideoCodec")
288
289     # Stage wasb_infer.py into the WASB repo src/ (exactly as the runners do) and import
290     # THAT copy, so its WASB imports and Hydra's configs/ dir resolve.
291     import shutil
292
293     from backend.pipeline.detectors import native_wasb_runner as nwr
294
295     src = osp.join(repo, "src")
296     staged = osp.join(src, "wasb_infer.py")
297     shutil.copy(nwr._INFER_PY, staged)
298     sys.path.insert(0, src)
299     try:
300         spec = importlib.util.spec_from_file_location("wasb_infer_staged", staged)
301         mod = importlib.util.module_from_spec(spec)
302         assert spec.loader is not None
303         spec.loader.exec_module(mod)
304
305         cpu_csv = str(tmp_path / "cpu_traj.csv")
306         gpu_csv = str(tmp_path / "gpu_traj.csv")
307         mod.run_video_streaming(video, weights, cpu_csv, fresh=True)
308         mod.run_video_nvdec_resident(video, weights, gpu_csv, fresh=True)
309
310         cpu, gpu = _load_traj(cpu_csv), _load_traj(gpu_csv)
311         common = sorted(set(cpu) & set(gpu))
312         assert len(common) >= 10, "fixture too short to be a meaningful gate"
313         agree = sum(1 for f in common if cpu[f][0] == gpu[f][0]) / len(common)
314         dists = [
315             float(np.hypot(cpu[f][1] - gpu[f][1], cpu[f][2] - gpu[f][2]))

```

```

316         for f in common
317             if cpu[f][0] == 1 and gpu[f][0] == 1
318         ]
319         # Validated on the L4 (#506): 96.3% visibility agreement, median 0.036 px.
320         assert agree >= 0.95, f"visibility agreement {agree:.3f} < 0.95"
321         assert dists, "no frames visible in both trajectories"
322         assert float(np.median(dists)) <= 1.0, (
323             f"median both-visible distance {float(np.median(dists)):.3f}px > 1.0px"
324         )
325     finally:
326         sys.path.remove(src)
327

```

3. user (2026-07-07T04:15:04.783Z)

```

44:from backend.config.models import IndexingConfig
45:from backend.pipeline.detectors.base import DetectorRunner
46:from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
49:from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

```

4. assistant (2026-07-07T04:15:15.032Z)

Now let me check the import chains and the deploy env dep lists.

5. user (2026-07-07T04:15:16.434Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     Unknown / typo'd config keys are rejected at load time via extra="forbid".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         pts_max_gap_seconds: float = Field(
31             default=10.0,
32             ge=0.0,
33             description=(
34                 "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                 "GoPro H.264/H.265 recordings typically have GOP intervals of 174s; "
36                 "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                 "recordings, and other encoders may use different GOP sizes ? tune "
38                 "this threshold for your recording device."

```

```
39         ),
40     )
```

6. user (2026-07-07T04:15:17.726Z)

```
11:imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
49:import argparse
50:import csv
51:import glob
52:import json
53:import logging
54:import os
55:import os.path as osp
56:from collections import defaultdict
57:from contextlib import contextmanager
```

7. user (2026-07-07T04:15:28.790Z)

```
=== backend/__init__.py ===
cat: 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/__init__.py':
No such file or directory
=== backend/pipeline/__init__.py ===
cat: 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/pipeline/__init__.py': No such file or directory
=== backend/pipeline/detectors/__init__.py ===
"""Low-level moving-object detectors that run outside the main Python process.
```

Houses:

- DetectorRunner ABC (the seam for WSL vs future Linux-native trajectory detectors).
- TrackNetRunner / WasbRunner (current WSL adapters; both inherit DetectorRunner).
- rally_gate (pure post-filter, no WSL).
- wasb_infer.py (the actual code that runs *inside* the WSL wasb env).

The ABC de-risks the WSL dependency (low-regret item 6 continuation): new runners can be added without changing hybrid segmenters or windowing logic. See base.py docstring for the exact contract + "how to add a runner" guide.

Keeping the heavy TF/torch deps quarantined in WSL (or a native subprocess) is deliberate: the main project env stays free of them.

```
from .base import DetectorRunner
```

Re-export the ABC so "from backend.pipeline.detectors import DetectorRunner" works

```
__all__ = ["DetectorRunner"]
```

The concrete runners are imported directly where needed (hybrids, eval/*).

This keeps heavy optional deps from being pulled at import time of the package.

```
=== backend/config/__init__.py ===
"""Configuration package.
```

Public API:

```
from backend.config import load_config, AppConfig, Config
```

This package is the canonical place for all configuration access.

```
import os
```

```
from .loader import (
    LIGHT_DEFAULT_CONFIG,
    load_config,
```



```

    save_default_config,
    write_light_default_config,
)
from .models import ( # re-export common ones
    AppConfig,
    Config,
    DeploymentConfig,
    IndexingConfig,
    StitchingConfig,
    StorageConfig,
    ValidationConfig,
)
from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
from .startup import (
    StartupCheck,
    StartupCheckError,
    enforce_startup_checks,
    run_startup_checks,
)

def default_db_path(config_path: str = "config.json") -> str:
    """Resolve the DB path from the typed config ? the single shared helper.

    Replaces the copy-pasted ``_default_db_path()`` in ``run_eval.py`` and
    ``run_regression.py``.
    """
    cfg = load_config(config_path)
    return os.path.join(cfg.output.output_dir, cfg.output.db_name)

__all__ = [
    "load_config",
    "save_default_config",
    "write_light_default_config",
    "LIGHT_DEFAULT_CONFIG",
    "default_db_path",
    "AppConfig",
    "Config",
    "DeploymentConfig",
    "IndexingConfig",
    "StitchingConfig",
    "StorageConfig",
    "ValidationConfig",
    "PROFILE_PRESETS",
    "suggest_profile",
    "probe_cuda_available",
    "StartupCheck",
    "StartupCheckError",
    "enforce_startup_checks",
    "run_startup_checks",
]

```

8. user (2026-07-07T04:15:29.468Z)

```

55:_INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
183:     shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))

```

9. user (2026-07-07T04:15:41.443Z)

```

69:import logging
70:import os
71:import shlex
72:import subprocess
73:from abc import ABC, abstractmethod

```

74:from typing import Any, Callable, List, Optional, Tuple

10. user (2026-07-07T04:15:42.742Z)

```
deploy\gcp\README.md:181:. .venv/bin/activate && pip install -e .[storage-gcs]
# GCS backend
deploy\cloudrun\Dockerfile:79:    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
deploy\cloudrun\Dockerfile:81:    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q
PyNvVideoCodec \
deploy\cloudrun\Dockerfile:82:    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
deploy\cloudrun\Dockerfile:103:# roboflow/trackers#474) so it's NOT pulled by `pip install .`;
install it out-of-band before
deploy\cloudrun\Dockerfile:106:RUN python3 -m pip install --upgrade pip \
deploy\cloudrun\Dockerfile:107:    && python3 -m pip install -r requirements-trackers.txt \
deploy\cloudrun\Dockerfile:108:    && python3 -m pip install ".[storage-gcs,cloud-run,auth]"
deploy\digitalocean\gpu_setup.sh:69:python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
deploy\digitalocean\gpu_setup.sh:71:python -m pip install -q PyNvVideoCodec
deploy\digitalocean\gpu_setup.sh:75:python -m pip install -q hydra-core omegaconf pandas opencv-
python-headless pillow scipy tqdm pyyaml matplotlib numpy || true
deploy\digitalocean\bootstrap.sh:29:python -m pip install --upgrade pip
deploy\digitalocean\bootstrap.sh:37: pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124
deploy\digitalocean\bootstrap.sh:39: pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
deploy\digitalocean\bootstrap.sh:47: pip install -r requirements-trackers.txt
deploy\digitalocean\bootstrap.sh:50: pip install -e .[dev]
deploy\gcp\nightly_regression\run_on_vm.sh:90: pip install -e .[storage-gcs]'
```

11. user (2026-07-07T04:15:50.439Z)

```
65     # is UNCHANGED.
66     #
67     # #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch
2.6.0+cu124
68     # (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11
baseline 0.582) +
69     # PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg
build and could
70     # not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version
shims this bump
71     # needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker).
numpy is
72     # UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no
requirements.txt ?
73     # the explicit list below is the (L4-validated) dependency closure its src/ imports
need.
74     ENV MAMBA_ROOT_PREFIX=/opt/conda
75     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
76         | tar -xj -C /usr/local bin/micromamba \
77         && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
78         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
79         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
80         torch==2.6.0 torchvision==0.21.0 --index-url
https://download.pytorch.org/whl/cu124 \
81         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
82         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
83         hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib numpy
84
85     # --- Main application env ---
86     WORKDIR /app
87     COPY pyproject.toml README.md ./
88     COPY requirements-trackers.txt ./
89     COPY backend ./backend
90     COPY training ./training
```

```

91      # Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker
PULLS the gs://
92      # input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-
storage is
93      # lazy-imported there); without it the job dies at the first gs:// fetch with
`ModuleNotFoundError:
94      # google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE,
which dispatches

```

12. user (2026-07-07T04:15:51.208Z)

```

55      # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms of
Service
56      # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail
on a fresh
57      # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
58      conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
>/dev/null 2>&1 || true
59      conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
>/dev/null 2>&1 || true
60
61      echo "[gpu] [3/6] WASB-SBDT repo"
62      if [ ! -d "$WASB_REPO/.git" ]; then
63          git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
64      fi
65
66      echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506
stack)"
67      conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.10
68      conda activate wasb
69      python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
70          --index-url https://download.pytorch.org/whl/cu124
71      python -m pip install -q PyNvVideoCodec
72      # WASB-SBDT ships NO requirements.txt: src/ imports hydra, pandas, etc. ? this explicit
list is
73      # the L4-validated dependency closure (#506). numpy is UNPINNED (the old "numpy<1.24"
was the
74      # torch-1.11 ABI pin); wasb_infer.py shims the NumPy-2.0 np.Inf removal for WASB's
tracker.
75      python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow scipy
tqdm pyyaml matplotlib numpy || true
76
77      echo "[gpu] [5/6] pretrained badminton weights"
78      WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
79      if [ -f "$WEIGHTS" ]; then
80          echo "[gpu]          found $WEIGHTS"
81      else
82          echo "[gpu]          !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
83      fi
84

```

13. assistant (2026-07-07T04:16:19.985Z)

Let me confirm native_wasb_runner's imports are module-level and unguarded.

14. user (2026-07-07T04:16:21.469Z)

```

30      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the

```

```

contract,
34     not the numbers.
35     """
36
37     import logging
38     import os
39     import shutil
40     import subprocess
41     from dataclasses import dataclass
42     from typing import List, Optional, Tuple
43
44     from backend.config.models import IndexingConfig
45     from backend.pipeline.detectors.base import DetectorRunner
46     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (

```

15. user (2026-07-07T04:17:12.536Z)

Structured output provided successfully